DATABASE SYSTEMS – LAB ASSIGNMENT 10

NAME: KESHAV. G

REG.NO: 24BCE1003

SLOT: L55+L56

PL/SQL CURSORS

Consider the table below:

```
CREATE TABLE EMP_CUR (
   EMP_ID NUMBER PRIMARY KEY,
   EMP_NAME VARCHAR2(30),
   SALARY NUMBER,
   DEPT_ID NUMBER
);

INSERT INTO EMP_CUR VALUES (101, 'Ravi', 50000, 10);
INSERT INTO EMP_CUR VALUES (102, 'Kiran', 60000, 20);
INSERT INTO EMP_CUR VALUES (103, 'Meena', 45000, 10);
INSERT INTO EMP_CUR VALUES (104, 'Arun', 70000, 30);
```

```
SQL> CREATE TABLE EMP_CUR (
  2       EMP_ID NUMBER PRIMARY KEY,
  3       EMP_NAME VARCHAR2(30),
  4       SALARY NUMBER,
  5       DEPT_ID NUMBER
  6  );

Table created.

SQL> INSERT INTO EMP_CUR VALUES (101, 'Ravi', 50000, 10);

1 row created.

SQL> INSERT INTO EMP_CUR VALUES (102, 'Kiran', 60000, 20);

1 row created.

SQL> INSERT INTO EMP_CUR VALUES (103, 'Meena', 45000, 10);

1 row created.

SQL> INSERT INTO EMP_CUR VALUES (104, 'Arun', 70000, 30);

1 row created.
```

```
SQL> SELECT * FROM EMP_CUR;

    EMP_ID EMP_NAME                          SALARY    DEPT_ID
---------- ------------------------------ ---------- ----------
       101 Ravi                                50000         10
       102 Kiran                               60000         20
       103 Meena                               45000         10
       104 Arun                                70000         30
```

PL/SQL Cursor Concepts:

- **Cursor** – A pointer to a query result set used to process multiple rows one at a time.
- **Explicit Cursor** – A cursor declared and controlled manually by the programmer.
- **DECLARE CURSOR** – Defines a cursor with a SELECT query.
- **OPEN Cursor** – Executes the query and allocates memory for the result set.
- **FETCH Cursor** – Retrieves one row at a time from the cursor into variables.
- **CLOSE Cursor** – Releases the memory occupied by the cursor.
- **Cursor Loop (LOOP…EXIT WHEN)** – Iterates through all rows returned by the cursor.
- **Cursor FOR Loop** – Implicit cursor that automatically handles OPEN, FETCH, and CLOSE.
- **%ROWTYPE with Cursor** – Stores an entire row fetched from the cursor.
- **%NOTFOUND** – Cursor attribute used to detect when no more rows are available.
- **%FOUND** – Indicates if the last FETCH was successful.
- **%ROWCOUNT** – Returns number of rows processed so far.
- **DML inside Cursor** – Allows INSERT/UPDATE/DELETE operations while iterating rows.
- **Implicit Cursor** – Automatically created by Oracle for SQL statements and FOR loops.

Demonstration:

SQL Query:

```
DECLARE

  -- Explicit Cursor
  CURSOR emp_cursor IS
    SELECT * FROM EMP_CUR;

  v_rec EMP_CUR%ROWTYPE;
  v_total NUMBER := 0;

BEGIN

  DBMS_OUTPUT.PUT_LINE('--- Cursor Processing ---');

  -- Open Cursor
  OPEN emp_cursor;

  LOOP
    -- Fetch row
    FETCH emp_cursor INTO v_rec;

    -- Exit condition
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Display record
    DBMS_OUTPUT.PUT_LINE(
      v_rec.EMP_ID || ' | ' ||
      v_rec.EMP_NAME || ' | ' ||
      v_rec.SALARY
    );

    -- Aggregate logic
    v_total := v_total + v_rec.SALARY;

    -- Conditional UPDATE inside cursor
    IF v_rec.SALARY < 50000 THEN
```

```
        UPDATE EMP_CUR
        SET SALARY = SALARY + 2000
        WHERE EMP_ID = v_rec.EMP_ID;

        DBMS_OUTPUT.PUT_LINE('Salary increased for: ' || v_rec.EMP_NAME);
      END IF;

    END LOOP;

    -- Close Cursor
    CLOSE emp_cursor;

    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || v_total);

    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('--- Updated Table ---');

    -- Cursor FOR loop (implicit cursor)
    FOR rec IN (SELECT * FROM EMP_CUR ORDER BY EMP_ID) LOOP
      DBMS_OUTPUT.PUT_LINE(
        rec.EMP_ID || ' | ' ||
        rec.EMP_NAME || ' | ' ||
        rec.SALARY
      );
    END LOOP;

END;
/
```

```
SQL> DECLARE
  2
  3      -- Explicit Cursor
  4      CURSOR emp_cursor IS
  5          SELECT * FROM EMP_CUR;
  6
  7      v_rec EMP_CUR%ROWTYPE;
  8      v_total NUMBER := 0;
  9
 10  BEGIN
 11
 12      DBMS_OUTPUT.PUT_LINE('--- Cursor Processing ---');
 13
 14      -- Open Cursor
 15      OPEN emp_cursor;
 16
 17      LOOP
 18          -- Fetch row
 19          FETCH emp_cursor INTO v_rec;
 20
 21          -- Exit condition
 22          EXIT WHEN emp_cursor%NOTFOUND;
 23
 24          -- Display record
 25          DBMS_OUTPUT.PUT_LINE(
 26              v_rec.EMP_ID || ' | ' ||
 27              v_rec.EMP_NAME || ' | ' ||
 28              v_rec.SALARY
 29          );
 30
 31          -- Aggregate logic
 32          v_total := v_total + v_rec.SALARY;
 33
```

```oracle_plsql

        -- Conditional UPDATE inside cursor
        IF v_rec.SALARY < 50000 THEN
            UPDATE EMP_CUR
            SET SALARY = SALARY + 2000
            WHERE EMP_ID = v_rec.EMP_ID;

            DBMS_OUTPUT.PUT_LINE('Salary increased for: ' || v_rec.EMP_NAME);
        END IF;

    END LOOP;

    -- Close Cursor
    CLOSE emp_cursor;

    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || v_total);

    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('--- Updated Table ---');

    -- Cursor FOR loop (implicit cursor)
    FOR rec IN (SELECT * FROM EMP_CUR ORDER BY EMP_ID) LOOP
        DBMS_OUTPUT.PUT_LINE(
            rec.EMP_ID || ' | ' ||
            rec.EMP_NAME || ' | ' ||
            rec.SALARY
        );
    END LOOP;

END;
/
```

Output:

```
--- Cursor Processing ---
101 | Ravi | 50000
102 | Kiran | 60000
103 | Meena | 45000
Salary increased for: Meena
104 | Arun | 70000
Total Salary: 225000
--- Updated Table ---
101 | Ravi | 50000
102 | Kiran | 60000
103 | Meena | 47000
104 | Arun | 70000

PL/SQL procedure successfully completed.
```